

LAPORAN JURNAL TOPIK ARSITEKTUR DAN DESAIN BASIS DATA

**Desain Polyglot Persistence Pada Sistem Informasi Akademik Yang
Dinamis**



Disusun oleh:

Doni Agus Setiawan	123140009
Tengku Hafid Diraputra	123140043
Andini Rahma Kemala	123140067
Zahwa Natasya Hamzah	123140069
Muhammad Ghama Al Fajri	123140182
Reyhan Oktavian Putra	123140202
Yuni Okta Safitri	123140213
Albi R. Suseno	120140095

Mata Kuliah: IF25-40405 - Manajemen Basis Data

Dosen Pengampu : Meida Cahyo Untoro, S.Kom., M.Kom.

**PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA**

2026

DAFTAR ISI

DAFTAR ISI.....	2
ABSTRAK.....	5
BAB I.....	1
PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Rumusan Masalah.....	1
1.3. Research gap.....	2
1.4. Tujuan penelitian.....	2
1.5. Kontribusi.....	3
1.6. Struktur Paper.....	3
1.6.1 Bab I Pendahuluan.....	3
1.6.2 Bab II Kajian Literatur.....	3
1.6.3 Bab III Metodologi.....	3
1.6.4 Bab IV Desain Eksperimen.....	3
1.6.5 Bab V Hasil yang Diharapkan.....	3
1.6.6 Bab VI Kesimpulan dan Saran.....	3
BAB II.....	4
KAJIAN LITERATUR.....	4
2.1 Klasifikasi Literatur.....	4
2.2 Tabel Perbandingan (sudah mencakup klasifikasi literatur).....	4
2.3. Identifikasi Research Gap & Posisi Penelitian.....	7
BAB III.....	9
METODOLOGI.....	9
3.1. Arsitektur Sistem / Framework.....	9
Gambar 3.1 Arsitektur Sistem Polyglot Persistence pada Sistem Informasi Akademik...	9
3.2 Desain Skema & Pemodelan Data.....	9
3.2.1 Skema Relasional PostgreSQL.....	10
3.2.2 Skema Dokumen MongoDB.....	10
3.2.3 Entity-Relationship Diagram (ERD) dan Pemetaan Antar-DBMS.....	11
3.2.4 Strategi Schema Evolution.....	11
3.3. Strategi Optimasi & Manajemen.....	12
3.3.1 Strategi Indexing.....	12
3.3.2 Query Tuning.....	12
3.3.3 Concurrency Control.....	12
3.3.4 Mekanisme Keamanan & Transaksi.....	13
3.3.5 Tools & Framework yang Digunakan.....	13
3.4. Metode Evaluasi.....	14
3.4.1 Skenario Evaluasi.....	14
3.4.2 Justifikasi.....	14

3.4.3 Parameter & Konfigurasi.....	15
3.4.4 Metrik Evaluasi.....	15
BAB IV.....	16
DESAIN EKSPERIMEN.....	16
4.1 Skenario Eksperimen.....	16
4.1.1. Skenario Data Transaksional (Relasional / OLTP).....	16
4.1.2. Skenario Data Aktivitas (Log VLE / Write-Heavy).....	16
4.1.3. Skenario Lonjakan Akses (High Concurrency Load).....	17
4.1.4. Skenario Pertumbuhan Data (Scalability).....	17
4.1.5. Skenario Perbandingan dengan Sistem Tunggal (Baseline).....	17
4.2 Baseline / Pembanding.....	18
4.2.1 Baseline Utama — PostgreSQL Monolitik.....	18
4.2.2 Baseline Sekunder — PostgreSQL dengan Optimasi.....	18
4.3 Environment Eksperimen.....	19
4.3.1 Spesifikasi Hardware (Konseptual).....	19
4.3.2 Spesifikasi Software dan Arsitektur Container.....	20
4.3.3 Dataset dan Persiapan Data.....	20
4.4 Metrik Evaluasi.....	22
4.4.1 Metrik Utama.....	22
4.4.2 Metrik Pendukung (Sumber Daya Sistem).....	23
4.4.3 Prosedur Pengukuran (Konseptual).....	23
BAB V.....	25
HASIL YANG DIHARAPKAN.....	25
5.1 Hipotesis Hasil.....	25
5.2 Hasil Eksperimen.....	25
5.2.1 Profil Environment Pengujian.....	25
5.2.2 Hasil Throughput.....	26
5.2.3 Hasil Latensi.....	26
5.2.4 Hasil Skenario Complex — Beban Tertinggi (FFF/2013J, 15 Detik).....	27
5.2.5 Profil CPU Usage — Skenario Complex.....	28
5.2.6 Kesesuaian Hasil dengan Hipotesis.....	28
Tabel 5.2 Kesesuaian Hasil dengan Hipotesis.....	29
5.3 Kontribusi yang Diharapkan.....	29
5.3.1 Bukti Empiris Keunggulan Polyglot pada Beban Skala Besar.....	29
5.3.2 Kerangka Kerja Pemetaan DBMS Berdasarkan Karakteristik Workload.....	29
5.3.3 Metodologi Benchmark Reproducible untuk Sistem Pendidikan Digital.....	29
5.3.4 Panduan Adopsi Polyglot untuk Institusi Pendidikan.....	29
BAB VI.....	31
KESIMPULAN DAN SARAN.....	31
6.1 Kesimpulan.....	31
6.2 Saran.....	31

DAFTAR PUSTAKA.....	32
LAMPIRAN.....	34
Tabel 1.1 Kontribusi Anggota.....	34
Tabel 1.2 Rencana Pengerjaan Tugas Besar.....	35

ABSTRAK

Perkembangan sistem informasi akademik berbasis digital menyebabkan peningkatan volume dan kompleksitas data yang dihasilkan, terutama dalam lingkungan pembelajaran daring. Data yang dikelola tidak hanya berupa data terstruktur seperti data mahasiswa dan nilai akademik, tetapi juga data aktivitas pembelajaran dalam jumlah besar yang bersifat semi-terstruktur. Kondisi ini menuntut adanya sistem basis data yang mampu menangani berbagai jenis data secara efisien dan fleksibel.

Penelitian ini mengusulkan desain arsitektur basis data menggunakan pendekatan *polyglot persistence*, yaitu kombinasi antara basis data relasional (SQL) dan non-relasional (NoSQL) dalam satu sistem. Dataset yang digunakan adalah Open University Learning Analytics Dataset (OULAD) yang mempresentasikan sistem pembelajaran daring dengan data dalam skala besar dan kompleks.

Metodologi yang digunakan meliputi perancangan arsitektur sistem, pemodelan skema basis data, serta penerapan strategi optimasi seperti indexing dan partitioning. Selain itu, dilakukan perancangan skenario benchmark untuk mengevaluasi performa sistem dalam menangani berbagai jenis query.

Hasil penelitian ini diharapkan dapat menghasilkan rancangan sistem basis data yang lebih skalabel, fleksibel, dan mampu mendukung kebutuhan sistem informasi akademik modern yang dinamis.

Kata Kunci : *Arsitektur Basis Data, Polyglot Persistence, Sistem Informasi Akademik, Cloud-Native Database, Schema Evolution, OULAD, Microservices, Large Language Models, SQL, NoSQL.*

BAB I

PENDAHULUAN

1.1. Latar Belakang

Transformasi digital di sektor pendidikan Indonesia mendorong Sistem Informasi Akademik (SIA) harus mampu mengelola data dalam jumlah yang semakin besar dan beragam. Data yang diolah tidak hanya berupa data terstruktur seperti nilai dan jadwal kuliah, tetapi juga data semi-terstruktur seperti log aktivitas mahasiswa, bahkan data tidak terstruktur seperti video materi pembelajaran [1]. Hal ini juga terlihat pada dataset Open University Learning Analytics Dataset (OULAD) yang berisi jutaan data aktivitas mahasiswa dalam sistem akademik saat ini harus mampu menangani berbagai jenis data dengan karakteristik yang berbeda-beda.

Namun, Penggunaan satu jenis basis data saja masih memiliki keterbatasan dalam menangani kondisi tersebut. Basis data relasional (SQL) memang cocok untuk data yang membutuhkan konsistensi tinggi, tetapi kurang fleksibel ketika harus menangani data dalam jumlah besar dan tidak terstruktur. Di sisi lain, basis data non-relasional (NoSQL) lebih fleksibel dan mampu menangani data besar, tetapi kurang optimal untuk transaksi yang kompleks [3]. Permasalahan ini menjadi lebih terasa pada kondisi nyata, misalnya saat pengisian KRS yang harus melayani banyak pengguna secara bersamaan, ditambah dengan aktivitas lain seperti akses materi dan pencatatan log secara real-time, sehingga dapat menyebabkan penurunan performa sistem.

Untuk mengatasi hal tersebut, pendekatan polyglot persistence mulai banyak digunakan dengan menggabungkan beberapa jenis basis data dalam satu sistem sesuai dengan kebutuhan data [9]. Pendekatan ini memungkinkan setiap jenis data disimpan pada basis data yang paling sesuai, sehingga sistem menjadi lebih efisien. Beberapa penelitian juga menunjukkan bahwa penggunaan polyglot persistence dapat meningkatkan performa sistem, terutama dalam menangani data dengan beban kerja yang beragam [5][6]. Selain itu, penggunaan arsitektur berbasis microservices juga mendukung sistem agar lebih fleksibel, termasuk dalam melakukan perubahan struktur data tanpa harus mengganggu layanan yang sedang berjalan [7][10]. Oleh karena itu, penelitian ini dilakukan untuk merancang arsitektur polyglot persistence yang sesuai untuk sistem informasi akademik yang bersifat dinamis.

1.2. Rumusan Masalah

Penelitian ini mencakup beberapa rumusan masalah, diantaranya:

1. Bagaimana merancang arsitektur *polyglot persistence* yang dapat mengatasi bottleneck akibat beban kerja yang beragam pada Sistem Informasi Akademik, terutama dalam menangani data transaksi dan log aktivitas secara bersamaan?
2. Bagaimana menentukan kombinasi Database Management System (DBMS) yang paling sesuai berdasarkan karakteristik workload pada sistem akademik termasuk

data terstruktur dan data berskala besar seperti pada dataset Open University Learning Analytics Dataset?

3. Bagaimana merancang mekanisme schema evolution agar sistem tetap fleksibel dalam menyesuaikan perubahan struktur data tanpa mengganggu performa layanan?

1.3. Research gap

Penelitian mengenai polyglot persistence sebenarnya sudah cukup banyak dilakukan. Namun, sebagian besar masih bersifat umum dan belum secara khusus membahas penerapannya pada Sistem Informasi Akademik (SIA), terutama yang memiliki beban kerja yang beragam dalam satu sistem [9]. Selain itu, pembahasan terkait integrasinya dengan pendekatan cloud-native juga masih terpisah-pisah dan belum benar-benar melihat kebutuhan sistem yang memiliki beban kerja yang beragam dalam satu arsitektur yang utuh [7][10]. Di sisi lain, penggunaan dataset berskala besar seperti Open University Learning Analytics Dataset (OULAD) menunjukkan bahwa sistem pembelajaran modern memiliki kombinasi data transaksi dan data log aktivitas dalam jumlah besar, namun belum banyak penelitian yang benar-benar memanfaatkan karakteristik data tersebut untuk merancang arsitektur polyglot persistence secara spesifik [5].

Selain itu, masih belum banyak penelitian yang memberikan kerangka kerja yang jelas dalam menentukan kombinasi DBMS yang tepat berdasarkan karakteristik workload [4]. Permasalahan lain yang juga jarang dibahas adalah terkait schema evolution pada sistem yang menggunakan lebih dari satu basis data [9], padahal hal ini penting karena sistem akademik cenderung mengalami perubahan struktur data secara berkala. Berdasarkan hal tersebut, masih terdapat celah penelitian dalam merancang arsitektur polyglot persistence berbasis cloud-native yang mampu menangani beban kerja yang beragam sekaligus tetap fleksibel terhadap perubahan skema.

1.4. Tujuan penelitian

Penelitian ini dilaksanakan untuk memenuhi beberapa tujuan utama, yaitu:

1. Merancang arsitektur polyglot persistence yang mampu mengatasi masalah bottleneck akibat beban kerja yang heterogen pada Sistem Informasi Akademik, khususnya dalam memproses data transaksional dan log aktivitas secara bersamaan.
2. Menentukan kombinasi Database Management System (DBMS) yang paling optimal dengan memetakan karakteristik workload sistem akademik, baik untuk data terstruktur maupun data berskala besar, menggunakan acuan dataset Open University Learning Analytics Dataset (OULAD).
3. Merancang mekanisme schema evolution yang dinamis agar sistem memiliki fleksibilitas tinggi dalam mengakomodasi perubahan struktur data tanpa mengorbankan performa maupun ketersediaan layanan operasional.

1.5. Kontribusi

Penelitian ini menawarkan solusi arsitektur untuk mengelola data pendidikan yang besar dan beragam menggunakan pendekatan polyglot persistence. Kontribusi penelitian ini adalah sebagai berikut:

1. Merancang arsitektur polyglot persistence yang mengintegrasikan basis data relasional (SQL) dan non-relasional (NoSQL) untuk menangani data transaksi akademik dan data log aktivitas pembelajaran dalam satu sistem
2. Mengusulkan pendekatan dalam menentukan kombinasi Database Management System (DBMS) yang sesuai berdasarkan karakteristik data dan workload pada sistem akademik.
3. Menerapkan konsep pemisahan proses baca dan tulis data dalam arsitektur berbasis microservices untuk meningkatkan performa sistem pada kondisi beban tinggi.
4. Mengoptimalkan pengelolaan data berskala besar seperti pada dataset Open University Learning Analytics Dataset sehingga sistem dapat menangani data secara lebih efisien dan skalabel.

1.6. Struktur Paper

1.6.1 Bab I Pendahuluan

Menjelaskan latar belakang, rumusan masalah, tujuan penelitian, serta research gap yang menjadi alasan penelitian ini.

1.6.2 Bab II Kajian Literatur

Menyajikan review terhadap jurnal-jurnal yang relevan dengan topik polyglot persistence dan manajemen basis data modern, dilengkapi dengan tabel perbandingan, klasifikasi literatur, serta identifikasi research gap sebagai dasar penelitian.

1.6.3 Bab III Metodologi

Menjelaskan metode perancangan yang digunakan dalam penelitian, meliputi perancangan arsitektur sistem, pemodelan data, serta pendekatan yang digunakan dalam membangun solusi berbasis polyglot persistence.

1.6.4 Bab IV Desain Eksperimen

Menguraikan hasil perancangan sistem yang diusulkan, meliputi desain arsitektur, struktur basis data, serta alur pengelolaan data dalam sistem polyglot persistence.

1.6.5 Bab V Hasil yang Diharapkan

Membahas hasil yang dibuat, termasuk kelebihan, kekurangan, serta potensi penerapan sistem dalam menangani beban kerja yang beragam pada Sistem Informasi Akademik.

1.6.6 Bab VI Kesimpulan dan Saran

Merangkum tujuan dan hasil yang diharapkan dari penelitian, serta memberikan saran pengembangan lebih lanjut untuk meningkatkan kualitas sistem di masa depan.

BAB II

KAJIAN LITERATUR

2.1 Klasifikasi Literatur

Berdasarkan kajian literatur, penelitian polyglot persistence dapat dikelompokkan ke beberapa kategori. Pertama, arsitektur sistem seperti microservices dan hybrid storage yang fokus pada fleksibilitas data [7][10][12][15]. Kedua, performa dan evaluasi yang membahas kinerja sistem dalam menangani data besar [3][5][6]. Ketiga, implementasi pada sistem nyata seperti social network dan big data, termasuk integrasi dengan teknologi modern [1][2][9]. Selain itu, terdapat kategori optimasi query [13] serta keamanan dan privasi data [14]. Klasifikasi ini menunjukkan masih adanya peluang pengembangan, terutama pada integrasi antar pendekatan dan penerapannya di bidang pendidikan.

2.2 Tabel Perbandingan (sudah mencakup klasifikasi literatur)

Tabel berikut merangkum perbandingan sistematis terhadap ketiga jurnal yang direview berdasarkan dimensi yang relevan dengan Pengantar Manajemen Basis Data.

Jurnal	Metode/Teknologi	Dataset	Hasil Utama	Kelebihan	Kekurangan
<i>Seabra & Lifschitz [2]</i>	Polyglot Big Data Processing menggunakan ekosistem Hadoop (HDFS, MapReduce) dikombinasikan dengan Spark, alat ingesti (Kafka, Storm), SQL engines (Hive), NoSQL (HBase, Cassandra), dan mediator (Apache Calcite).	Menggunakan 4 studi kasus teoretis industri: Kesehatan (IoT medis), Pasar Saham (<i>real-time feed</i>), Jejaring Sosial (Twitter), dan <i>Smart Cities</i> .	Arsitektur polyglot terbukti fleksibel, mampu memetakan alat yang tepat untuk tugas spesifik (misal: Spark untuk analitik <i>real-time</i> , Hive untuk <i>batch</i>) dengan HDFS sebagai pusat penyimpanan (<i>Data Lake</i>).	Optimalisasi performa sesuai jenis data, skalabilitas tinggi untuk Big Data, memiliki toleransi kesalahan (fault tolerance) yang kuat.	Kompleksitas integrasi di lapisan aplikasi sangat tinggi, butuh middleware perantara, belum ada uji coba/eksperimen metrik dunia nyata dalam makalah ini.
<i>Roy-Hubara et al. [4]</i>	Metode RbDSM (Requirement-based Database Selection Method) untuk memilih database berdasarkan kebutuhan data, fungsional, dan non-fungsional, dengan pendekatan pembobotan dan evaluasi multi-kriteria.	Menggunakan 3 studi kasus (tool pengembangan, knowledge management, dan industri) berupa kebutuhan sistem, bukan dataset numerik.	Rekomendasi database sesuai kebutuhan aplikasi, Mendukung konsep polyglot persistence, Metode valid melalui studi kasus, Bobot kriteria mempengaruhi hasil.	Sistematis dan Terstruktur, Relevan untuk arsitektur modern, Pertimbangan lengkap (data & sistem), Validasi studi kasus.	Kompleks dalam implementasi, Kurang detail teknis DBMS, Tidak ada dataset besar, Bergantung pada subjektivitas bobot.
<i>Calatrava et al. [6]</i>	Mengusulkan pendekatan holistic scalability berbasis Cascading	Menggunakan data time series (sensor data) berupa aliran	Mengurangi ukuran cluster hingga 33%–50%,	Efisien dalam penggunaan resource	Kompleksitas arsitektur tinggi, Implementasi

	Polyglot Persistence, dengan pembagian sistem menjadi beberapa data model (DM1, DM2, DM3) serta penggunaan teknik sharding, replication, dan cluster database untuk meningkatkan efisiensi dan skalabilitas.	data kontinu (timestamp, sensor ID, value) yang diuji dalam berbagai skenario cluster untuk mengevaluasi performa dan skalabilitas sistem.	Skalabilitas mencapai >85% dari ideal, Mampu menangani hingga 250.000 data/detik, Lebih efisien dalam penggunaan resource, Tetap menjaga keamanan data (replication).	(cost-saving), Skalabilitas tinggi untuk big data, Arsitektur fleksibel dan modular, Performa tinggi untuk data real-time, Cocok untuk sistem distributed.	sulit (butuh konfigurasi detail), Bergantung pada skenario penggunaan, Fokus hanya pada time-series database, Masih terbatas pada studi eksperimental.
<i>Ye et al. [5]</i>	Mengusulkan penggunaan MDBench untuk evaluasi basis data multi-model tunggal melawan arsitektur <i>polyglot persistence</i> , mengeksekusi 4 kelompok eksperimen yang mencakup <i>multi-threading</i> , kueri gabungan lintas model, pengujian keandalan, dan ketersediaan sistem.	Menggunakan kumpulan data pengujian multi-model hasil <i>generate</i> sintesis menggunakan <i>scalable multi-model data generator</i> rancangan peneliti sendiri.	ArangoDB (<i>Multi-model</i>) lebih unggul dalam operasi penyisipan (<i>insertion</i>) data berjenis graf, sedangkan <i>Polyglot Persistence</i> (MongoDB + Neo4j) lebih unggul dalam penghapusan data berbasis dokumen dan kueri asosiatif ke banyak tabel.	Menyediakan rancangan benchmark komprehensif yang tidak hanya mengukur waktu eksekusi tetapi juga menyoroti jejak penggunaan daya dan metrik non fungsional dengan algoritma generator data yang dirancang stabil untuk pengujian data skala masif.	Eksperimen sangat terikat pada contoh teknologi tertentu dan mungkin saja akan memberikan hasil yang berbeda secara signifikan jika pengujian dilakukan dengan menggunakan <i>Database Management System</i> (DBMS) modern lain.
<i>Kazanavi ċius et al. [10]</i>	Mengusulkan metode migrasi database dari monolith ke microservices berbasis polyglot persistence, dengan tahapan analisis sistem lama, pemecahan microservices, pengembangan model data baru, serta transformasi data menggunakan repository layer dan REST API.	Menggunakan studi kasus sistem mainframe (legacy system) dengan database monolith sebagai sumber data, yang kemudian ditransformasikan ke multi-model polyglot persistence. Dataset berupa data nyata dari sistem lama.	Berhasil melakukan migrasi dari monolith ke microservices, Meningkatkan kualitas data (consistency, availability, portability), Mendukung arsitektur distributed system, Polyglot persistence lebih fleksibel dibanding monolith, Metode terbukti melalui proof-of-concept.	Memberikan langkah migrasi yang jelas dan sistematis, Relevan untuk modernisasi sistem legacy, Menggunakan studi kasus nyata, Mendukung arsitektur microservices, Meningkatkan kualitas data.	Kompleksitas migrasi tinggi, Proses transformasi data sulit, Bergantung pada kualitas database lama, Implementasi membutuhkan effort besar, Studi kasus terbatas pada satu sistem.

<i>Calatrava et al. [8]</i>	Menggunakan metode <i>Cascading Polyglot Persistence</i> , yaitu menyimpan data secara bertahap dalam tiga jenis basis data: <i>key-value</i> (harian), <i>short-column</i> (bulanan), dan <i>long-column</i> (jangka panjang).	Simulasi 500 sensor yang mencatat data setiap menit selama 10 tahun (lebih dari 2,6 miliar data).	Kecepatan pencarian data hingga 12 kali lipat lebih cepat dari MongoDB dan 5 kali lebih cepat dari InfluxDB. Kecepatan asupan data (<i>ingestion</i>) juga 2 kali lipat lebih tangkas	Kinerja sangat cepat dan sangat hemat ruang penyimpanan (hingga 50%) dibandingkan sistem basis data biasa	Sistem lebih rumit untuk dibangun secara teknis dan kinerjanya menjadi tidak efisien jika digunakan di luar fungsi utamanya (misal: mencari riwayat jangka panjang pada model harian)
<i>de Curtò et al. [1]</i>	Menggunakan beberapa basis data sekaligus (MongoDB, Neo4j, Redis) dan LLM (<i>Google Gemini</i>) untuk mengubah pertanyaan bahasa biasa menjadi kueri basis data.	Data simulasi jaringan sosial dengan 100 profil pekerja dan 500 hubungan.	LLM mampu menerjemahkan kueri dengan akurasi hingga 100%, dan semua kueri berhasil dijalankan setelah ditambahkan sistem cadangan saat terjadi kegagalan.	Pengguna bisa mengambil data tanpa perlu memahami pemrograman, dan sistem tetap berjalan meskipun ada masalah koneksi.	LLM masih kesulitan memahami pertanyaan yang tidak jelas, waktu pemrosesan sedikit lebih lambat, dan perlu pelatihan khusus (<i>fine-tuning</i>) agar lebih akurat.
<i>Lajam & Mohamm ad [9]</i>	Menggunakan metodologi <i>Literature Review</i> komprehensif untuk mengkaji studi-studi baru terkait <i>polyglot persistence</i> dan mengklasifikasikan arsitektur implementasi menjadi 4 kategori yaitu <i>Application-coordinated</i> , <i>Service-oriented</i> , <i>Polyglot-Persistence-as-a-Service</i> (PPaaS), dan <i>Multi-model Databases</i> .	Merupakan <i>review paper</i> , sehingga tidak menggunakan dataset numerik atau kuantitatif, melainkan menggunakan kumpulan data berupa puluhan studi dan literatur terdahulu yang berkaitan dengan tantangan dan implementasi sistem basis data terdistribusi.	Menghasilkan analisis mendalam mengenai isu-isu yang ada pada tiap kategori <i>polyglot persistence</i> dan mengusulkan strategi implementasi praktis sebagai panduan keputusan desai yang lebih bijak dalam mengatasi masalah kompleksitas sistem penyimpanan hybrid.	Mengatasi kebingungan arsitektural yang ada pada studi-studi sebelumnya dengan menyediakan pengelompokan yang jelas dan sangat membantu sebagai perbandingan pendekatan satu dengan yang lainnya.	Kajian bersifat teoritis konseptual dan tidak memberikan hasil eksperimen teknis, uji coba lapangan, atau benchmark metrik performa secara nyata pada skenario tertentu.
<i>Van Landuyt et al. [3]</i>	Menggunakan pendekatan benchmarking (MDBench) untuk membandingkan database multi-model (ArangoDB) dengan polyglot persistence (MongoDB + Neo4j).	Dataset sintetis multi-model yang dihasilkan menggunakan data generator untuk simulasi skala besar.	Multi-model lebih unggul pada operasi tertentu seperti insertion graf, sedangkan polyglot persistence lebih baik dalam query	Memberikan evaluasi performa yang cukup lengkap dan realistis, serta mencakup berbagai	Hasil sangat bergantung pada DBMS yang digunakan, sehingga mungkin berbeda jika

	Pengujian meliputi multi-threading, query lintas model, dan reliability.		kompleks dan penghapusan data.	skenario pengujian.	menggunakan teknologi lain.
Ahaidous et al. [7]	Mengusulkan arsitektur hybrid storage berbasis microservices dengan pendekatan polyglot persistence untuk mengelola berbagai jenis data	Menggunakan pendekatan konseptual (arsitektur sistem), tidak fokus pada dataset numerik tertentu.	Arsitektur yang diusulkan mampu meningkatkan fleksibilitas sistem dalam mengelola data yang beragam dan mendukung sistem terdistribusi.	Fleksibel, scalable, dan cocok untuk sistem modern dengan data heterogen.	Tidak terdapat evaluasi performa secara detail, lebih fokus pada desain arsitektur.

Tabel 2.1 Perbandingan Jurnal

2.3. Identifikasi Research Gap & Posisi Penelitian

Konsep *polyglot persistence* telah terbukti efektif mengatasi masalah skalabilitas pada sistem basis data relasional. Namun, mayoritas studi terdahulu masih berfokus pada domain aplikasi komersial secara umum, sehingga menyisakan kesenjangan penelitian (*research gap*) dalam konteks Sistem Informasi Akademik (SIA):

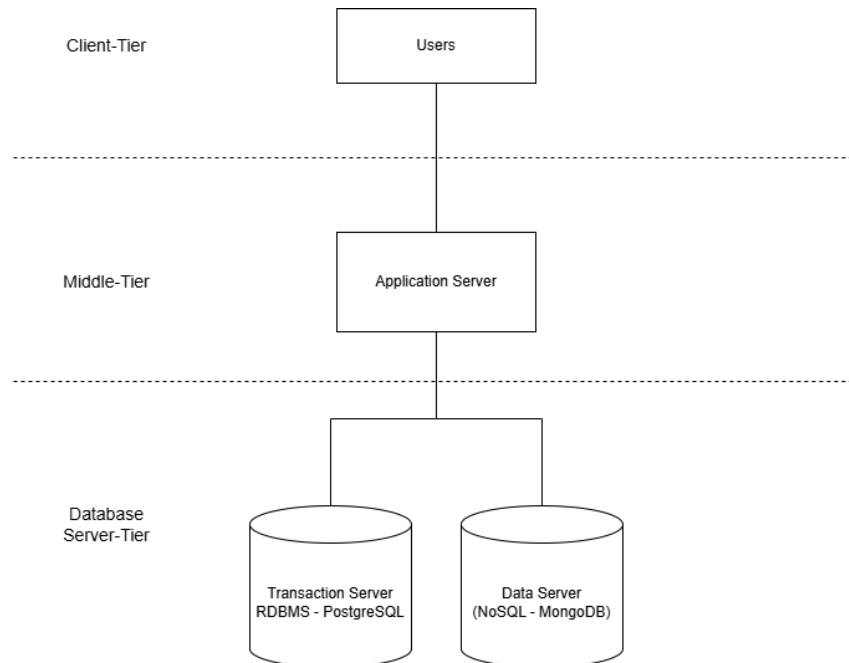
1. Belum adanya kerangka kerja pemilihan kombinasi DBMS yang spesifik: Studi saat ini belum memetakan beban kerja SIA yang unik, yaitu keharusan menangani data transaksional yang kaku secara bersamaan dengan pemrosesan log aktivitas *e-learning* yang sangat masif dan dinamis.
2. Keterbatasan mekanisme *schema evolution* lintas model: Belum banyak penelitian yang membahas bagaimana struktur data pada arsitektur hibrida dapat beradaptasi terhadap perubahan (seperti pergantian kurikulum) tanpa mengorbankan performa atau menyebabkan waktu henti (*downtime*).

Penelitian ini memposisikan diri untuk mengisi celah tersebut dengan merancang arsitektur referensi polyglot persistence yang diadaptasikan khusus untuk SIA. Berbeda dengan studi sebelumnya yang menggunakan metrik generik, penelitian ini memetakan kombinasi DBMS dan merancang mekanisme *schema evolution* menggunakan data dunia nyata (Open University Learning Analytics Dataset/OULAD) untuk memastikan sistem bebas dari bottleneck saat menghadapi beban kerja yang beragam.

BAB III

METODOLOGI

3.1. Arsitektur Sistem / Framework



Gambar 3.1 Arsitektur Sistem Polyglot Persistence pada Sistem Informasi Akademik

Diagram pada Gambar 3.1 menunjukkan arsitektur sistem yang digunakan, yaitu arsitektur client-server dengan model three-tier, dimana komponen Users akan bertindak sebagai front-end dan tidak memiliki akses panggilan ke database secara langsung. Users berkomunikasi dengan Application Server melalui tampilan antarmuka, dan Application Server berkomunikasi dengan sistem database di Database Server-tier untuk mengakses data. Komponen Application Server sendiri berisikan logika-logika bisnis dan pengatur lalu lintas data dalam sistem. pada Database Server-Tier, digunakan praktik Polyglot Persistence, yaitu praktik menggunakan beberapa teknologi database yang berbeda dalam satu sistem untuk menyimpan data-data sesuai karakteristiknya yang terbaik untuk mengurangi beban kerja [16]. Dua teknologi database tersebut yaitu untuk Transaction Server menggunakan Relational Database (RDMS) dan untuk Data Server menggunakan NoSQL.

Transaction Server mengurus data administrative (studentInfo, StudentRegistration, StudentAssessments, assessments, courses) yang bersifat relational dan konsisten. Sementara Data Server mengurus data logs interaktif (studentVle, vle) yang bersifat semi-terstruktur dan fleksibel.

3.2 Desain Skema & Pemodelan Data

Perancangan skema data dilakukan dengan membagi data berdasarkan karakteristiknya, yaitu data terstruktur dan semi-terstruktur. Data akademik disimpan pada PostgreSQL,

sedangkan data aktivitas pembelajaran disimpan pada MongoDB. Pendekatan ini mengikuti konsep polyglot persistence yang memungkinkan penggunaan beberapa jenis database dalam satu sistem agar lebih optimal dalam menangani beban kerja yang berbeda [9][4].

3.2.1 Skema Relasional PostgreSQL

PostgreSQL digunakan untuk menyimpan data utama dalam sistem akademik, seperti data mahasiswa, mata kuliah, registrasi, dan penilaian. Tabel students menyimpan informasi mahasiswa, sedangkan tabel courses menyimpan data mata kuliah beserta periode pembelajarannya.

Relasi antara mahasiswa dan mata kuliah dicatat pada tabel student_registrations. Sementara itu, tabel assessments digunakan untuk menyimpan komponen penilaian, dan tabel student_assessments menyimpan nilai mahasiswa. Struktur relasional ini dirancang untuk mendukung proses transaksi seperti pengisian KRS dan pengolahan nilai, sekaligus menjaga konsistensi data melalui penggunaan primary key dan foreign key [5].

```
Schema PostgreSQL: Data Transaksional Akademik
CREATE TABLE courses (
  code_module VARCHAR(10),
  code_presentation VARCHAR(10),
  PRIMARY KEY (code_module, code_presentation)
);
CREATE TABLE students (
  id_student INT PRIMARY KEY
);
CREATE TABLE student_registrations (
  id_student INT,
  code_module VARCHAR(10),
  code_presentation VARCHAR(10),
  PRIMARY KEY (id_student, code_module, code_presentation)
);
CREATE TABLE assessments (
  id_assessment SERIAL PRIMARY KEY
);
CREATE TABLE student_assessments (
  id_student INT,
  id_assessment INT,
  PRIMARY KEY (id_student, id_assessment)
);
```

3.2.2 Skema Dokumen MongoDB

MongoDB digunakan untuk menyimpan data log aktivitas mahasiswa selama proses pembelajaran. Data ini berasal dari interaksi mahasiswa dengan sistem e-learning dan memiliki volume yang besar serta terus bertambah.

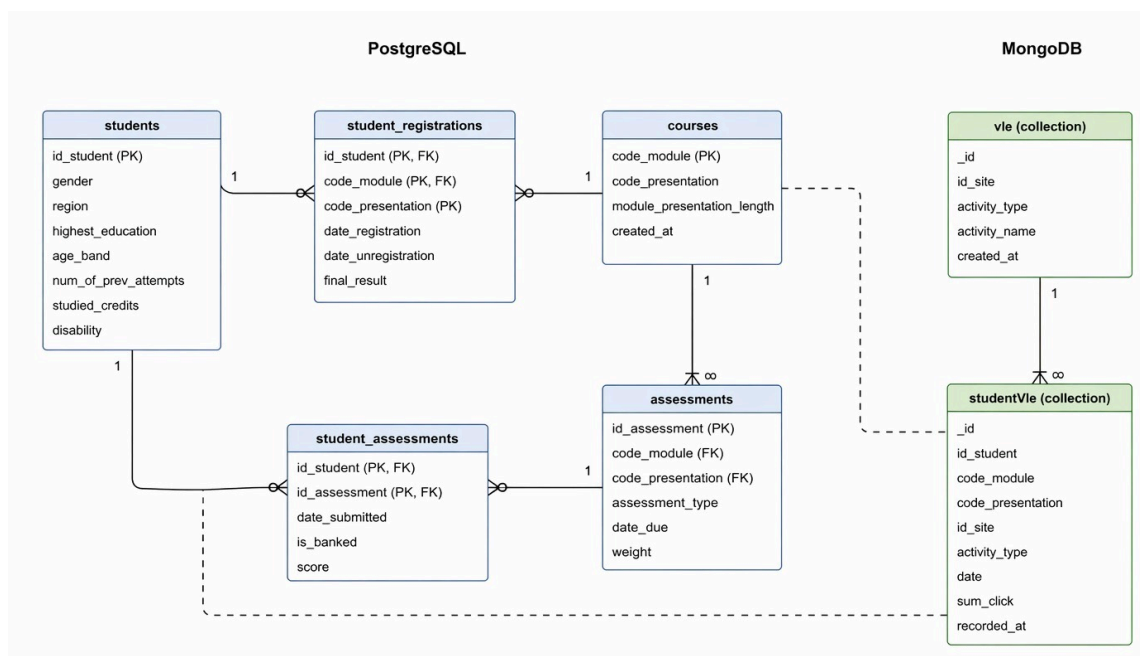
Setiap aktivitas disimpan dalam bentuk dokumen tanpa relasi seperti pada database relasional. Field seperti `id_student` dan `code_module` tetap digunakan sebagai referensi untuk menghubungkan data dengan PostgreSQL saat diperlukan. Pendekatan ini sesuai untuk data dengan karakteristik write-heavy dan mendukung skalabilitas yang lebih baik [6][3].

```
Schema Mongoose: Log Aktivitas VLE (MongoDB)
const vleActivitySchema = {
  id_student: Number,
  code_module: String,
  code_presentation: String,
  date: Number,
  sum_click: Number
};
```

3.2.3 Entity-Relationship Diagram (ERD) dan Pemetaan Antar-DBMS

ERD digunakan untuk menggambarkan hubungan antar entitas pada PostgreSQL serta keterkaitannya dengan MongoDB. Relasi pada PostgreSQL dilakukan secara langsung menggunakan foreign key untuk menjaga integritas data.

Sementara itu, hubungan dengan MongoDB bersifat logis, bukan relasi langsung. Data pada MongoDB tetap menyimpan atribut seperti `id_student` dan `code_module` agar dapat dikaitkan dengan data akademik. Pendekatan ini umum digunakan dalam arsitektur polyglot persistence untuk menjaga performa tanpa mengorbankan fleksibilitas [15][13].



Gambar 3.4. ERD Lengkap Sistem Polyglot Persistence (PostgreSQL + MongoDB)

3.2.4 Strategi Schema Evolution

Sistem dirancang agar tetap dapat menyesuaikan perubahan struktur data. Pada PostgreSQL, perubahan skema dilakukan secara bertahap agar tidak mengganggu sistem yang sedang berjalan.

Sementara itu, MongoDB lebih fleksibel karena tidak terikat skema tetap. Perubahan struktur dokumen dapat dilakukan tanpa harus memodifikasi seluruh data yang sudah ada. Pendekatan ini mendukung kebutuhan sistem yang dinamis, terutama dalam lingkungan pendidikan yang sering mengalami perubahan [10][7].

3.3. Strategi Optimasi & Manajemen

Strategi optimasi dilakukan dengan menyesuaikan peran masing-masing database dalam arsitektur polyglot persistence. PostgreSQL difokuskan untuk mengelola data akademik yang membutuhkan konsistensi tinggi, sedangkan MongoDB digunakan untuk menangani data log aktivitas yang berjumlah besar. Pendekatan ini memungkinkan sistem bekerja lebih efisien karena setiap jenis data diproses oleh database yang paling sesuai [9][4].

Selain itu, pembagian beban kerja melalui application layer membantu mengurangi bottleneck saat sistem menangani akses secara bersamaan. Konsep ini juga sejalan dengan pendekatan arsitektur berbasis microservices yang memisahkan tanggung jawab sistem menjadi lebih spesifik [7][10].

3.3.1 Strategi Indexing

Indexing digunakan untuk mempercepat proses pencarian data pada kedua database. Pada PostgreSQL, index diterapkan pada kolom yang sering digunakan dalam query, seperti `id_student`, `code_module`, dan `code_presentation`. Hal ini penting untuk meningkatkan performa query pada data transaksional akademik [5].

Pada MongoDB, index diterapkan pada field seperti `id_student` dan `date` untuk mendukung pencarian data log. Selain itu, digunakan juga mekanisme TTL (Time-To-Live) untuk menghapus data lama secara otomatis agar penyimpanan tetap efisien. Strategi ini sesuai dengan pengelolaan data time-series dalam sistem berskala besar [6].

3.3.2 Query Tuning

Query tuning dilakukan agar proses pengambilan data berjalan lebih efisien. Pada PostgreSQL, optimasi dilakukan dengan memperhatikan struktur query dan penggunaan join antar tabel, terutama pada proses analisis data akademik.

Sementara itu, pada MongoDB, query disesuaikan dengan struktur dokumen sehingga data dapat diambil tanpa operasi yang kompleks. Pendekatan ini membantu mengurangi beban sistem saat memproses data dalam jumlah besar [3].

Dalam implementasinya, strategi query juga dapat dikembangkan dengan pendekatan lintas database, di mana data dari PostgreSQL dan MongoDB digabungkan pada level aplikasi untuk kebutuhan analisis yang lebih kompleks [13].

3.3.3 Concurrency Control

Pengelolaan akses data dilakukan untuk menjaga kestabilan sistem saat banyak pengguna mengakses secara bersamaan. PostgreSQL menggunakan mekanisme transaksi untuk memastikan konsistensi data tetap terjaga.

Di sisi lain, MongoDB memungkinkan proses penulisan data dilakukan secara paralel pada banyak dokumen. Hal ini membuat MongoDB lebih cocok untuk menangani data log aktivitas yang memiliki frekuensi penulisan tinggi [9].

Pendekatan ini penting terutama pada sistem akademik yang sering mengalami lonjakan akses, seperti saat pengisian KRS atau aktivitas pembelajaran berlangsung secara bersamaan.

3.3.4 Mekanisme Keamanan & Transaksi

Keamanan sistem dilakukan dengan membatasi akses pengguna berdasarkan perannya. Setiap pengguna hanya dapat mengakses data sesuai kebutuhan, sehingga risiko penyalahgunaan data dapat dikurangi.

Selain itu, data log yang disimpan pada MongoDB dikelola dengan mekanisme penghapusan otomatis untuk menjaga efisiensi penyimpanan. Pendekatan ini juga dapat dikombinasikan dengan teknik perlindungan data untuk menjaga kerahasiaan informasi pengguna [14].

Pengelolaan data secara keseluruhan dilakukan agar sistem tetap stabil dan mampu menangani pertumbuhan data secara berkelanjutan.

3.3.5 Tools & Framework yang Digunakan

Tabel berikut merangkum seluruh tools dan framework yang digunakan dalam strategi optimasi dan manajemen sistem berdasarkan karakteristik dataset OULAD:

Komponen	Tools/Framework	Fungsi
Database Relasional (SQL)	PostgreSQL	Menyimpan data akademik terstruktur seperti mahasiswa, mata kuliah, dan penilaian
Database non-relasional (NoSQL)	MongoDB	Menyimpan data log aktivitas pembelajaran dalam jumlah besar
Application Layer	Node.js / API Layer	Menghubungkan client dengan database
ORM / ODM	Prisma / Mongoose	Mempermudah pengelolaan query dan skema

Containerization	Docker	Menjalankan sistem secara terisolasi
Monitoring	Built-in DB Tools	Memantau performa database

Tabel 3.1 Tabel Tools dan Framework Sistem OULAD.

Tabel di atas merangkum tools dan framework yang digunakan dan dipilih berdasarkan kebutuhan sistem yang menggabungkan dua jenis database. PostgreSQL dan MongoDB menjadi komponen utama dalam penyimpanan data, sedangkan application layer berfungsi sebagai penghubung antara pengguna dan database. Penggunaan ORM/ODM membantu dalam pengelolaan data agar lebih terstruktur, sementara Docker digunakan untuk memastikan sistem dapat dijalankan secara konsisten di berbagai lingkungan. Pemilihan tools ini sejalan dengan pendekatan arsitektur modern yang fleksibel dan mudah dikembangkan [7][2].

3.4. Metode Evaluasi

Evaluasi pada penelitian ini dilakukan secara konseptual dengan menganalisis bagaimana arsitektur polyglot persistence yang dirancang mampu menangani berbagai jenis beban kerja pada sistem akademik. Analisis dilakukan dengan membandingkan pendekatan ini terhadap sistem basis data tunggal berdasarkan aspek skalabilitas, fleksibilitas, dan efisiensi pengelolaan data.

Pendekatan evaluasi mengacu pada karakteristik dataset OULAD yang memiliki kombinasi data transaksional dan data log aktivitas dalam jumlah besar. Dengan demikian, evaluasi difokuskan pada kesesuaian desain arsitektur dalam menangani kedua jenis data tersebut secara bersamaan [3][5].

3.4.1 Skenario Evaluasi

Evaluasi dilakukan melalui beberapa skenario yang merepresentasikan kebutuhan umum pada sistem informasi akademik.

Skenario pertama adalah pengolahan data transaksional, yang mencakup pengambilan data mahasiswa, registrasi, dan penilaian pada PostgreSQL. Skenario ini menggambarkan bagaimana sistem menangani operasi dasar seperti insert, update, dan select pada data akademik.

Skenario kedua adalah pengolahan data analitis, yang melibatkan perhitungan nilai rata-rata, jumlah mahasiswa, dan distribusi hasil belajar. Skenario ini digunakan untuk melihat bagaimana sistem mendukung kebutuhan pelaporan akademik.

Skenario ketiga adalah pengolahan data log aktivitas, yang berfokus pada data interaksi mahasiswa pada sistem pembelajaran yang disimpan di MongoDB. Skenario ini menggambarkan pengolahan data dalam jumlah besar dan berbasis waktu.

Skenario keempat adalah integrasi data (hybrid query), yaitu penggabungan data akademik dari PostgreSQL dengan data aktivitas dari MongoDB untuk analisis yang lebih lengkap.

Skenario terakhir adalah skalabilitas sistem, yang menggambarkan bagaimana arsitektur mampu menangani peningkatan jumlah data dan pengguna berdasarkan karakteristik dataset OULAD.

3.4.2 Justifikasi

Setiap skenario yang digunakan disusun untuk merepresentasikan kebutuhan nyata dalam sistem akademik. Skenario transaksi digunakan untuk memastikan pengelolaan data akademik tetap konsisten, sedangkan skenario analitis mencerminkan kebutuhan pelaporan. Skenario log aktivitas digunakan karena dataset OULAD memiliki volume data yang besar, sehingga membutuhkan pendekatan yang berbeda dari data relasional. Skenario integrasi menjadi penting karena sistem akademik modern tidak hanya mengolah data utama, tetapi juga menggabungkannya dengan data aktivitas pembelajaran. Sementara itu, skenario skalabilitas digunakan untuk menunjukkan bahwa arsitektur yang dirancang mampu menyesuaikan diri terhadap pertumbuhan data dan pengguna, yang merupakan karakteristik umum sistem berbasis digital.

3.4.3 Parameter & Konfigurasi

Parameter evaluasi didasarkan pada karakteristik data dan pola akses yang terdapat pada dataset OULAD. Parameter utama yang digunakan meliputi jenis data (transaksional dan log), volume data, serta pola akses data.

Konfigurasi sistem mengikuti arsitektur yang telah dirancang, yaitu penggunaan PostgreSQL untuk data akademik dan MongoDB untuk data aktivitas. Evaluasi difokuskan pada bagaimana kedua sistem ini bekerja sesuai dengan perannya masing-masing, bukan pada pengujian teknis secara langsung.

3.4.4 Metrik Evaluasi

Evaluasi dilakukan dengan menggunakan beberapa metrik utama yang umum digunakan dalam analisis sistem basis data, yaitu:

- Waktu respon (response time) untuk melihat kecepatan sistem dalam memproses data
- Skalabilitas untuk menilai kemampuan sistem dalam menangani peningkatan data
- Efisiensi pengelolaan data untuk melihat kesesuaian penggunaan masing-masing DBMS
- Konsistensi data untuk memastikan data tetap akurat pada sistem relasional

Metrik ini digunakan untuk menilai apakah arsitektur yang dirancang sudah sesuai dengan kebutuhan sistem akademik yang memiliki data beragam.

BAB IV

DESAIN EKSPERIMEN

4.1 Skenario Eksperimen

Skenario eksperimen dalam penelitian ini dibuat berdasarkan ciri-ciri data yang ada di Open University Learning Analytics Dataset (OULAD), yang terbagi menjadi dua jenis utama, yaitu data transaksi akademik dan data aktivitas belajar. Dataset ini terdiri dari beberapa tabel utama seperti studentInfo, courses, studentRegistration, assessments, studentAssessment, dan studentVle, masing-masing memiliki struktur serta cara akses data yang berbeda. Perbedaan tersebut digunakan sebagai dasar dalam merancang skenario eksperimen. Tujuan dari eksperimen ini adalah untuk menganalisis secara konseptual bagaimana arsitektur polyglot persistence dapat menangani berbagai jenis beban kerja dalam Sistem Informasi Akademik. Skenario ini tidak diterapkan langsung, tetapi digunakan sebagai acuan untuk mengevaluasi kondisi operasional yang biasa terjadi dalam sistem akademik.

4.1.1. Skenario Data Transaksional (Relasional / OLTP)

Skenario ini menggunakan tabel studentInfo, studentRegistration, assessments, dan studentAssessment yang saling berhubungan erat dan digunakan dalam proses akademik seperti mengelola data mahasiswa, mendaftarkan mata kuliah, serta menghitung nilai akhir. Berdasarkan dataset OULAD, tabel studentRegistration dan studentInfo masing-masing memiliki sekitar 32.593 baris data yang menunjukkan aktivitas akademik selama satu periode. Karakteristik data pada skenario ini adalah:

1. terstruktur
2. membutuhkan konsistensi tinggi
3. sering melibatkan operasi JOIN

Skenario ini dibuat agar bisa meniru beban transaksi yang terjadi pada proses pendaftaran mahasiswa, seperti pengisian KRS, perubahan data, dan pengambilan informasi akademik secara bersamaan. Dalam desain sistem yang diajukan, data tersebut dikelola dengan menggunakan PostgreSQL agar data tetap utuh dan konsisten. Skenario ini berfungsi sebagai dasar untuk menganalisis beban kerja transaksi berbasis relasi [4].

4.1.2. Skenario Data Aktivitas (Log VLE / Write-Heavy)

Skenario ini berpusat pada tabel studentVle yang menyimpan data aktivitas mahasiswa dalam jumlah yang sangat banyak, yaitu lebih dari 10 juta baris. Data ini mencatat cara mahasiswa berinteraksi dengan sistem belajar, termasuk jumlah klik yang dilakukan, durasi waktu akses, dan tipe aktivitas yang dilakukan. Karakteristik data pada skenario ini adalah:

1. volume data sangat besar
2. tidak membutuhkan relasi kompleks
3. bersifat terus bertambah (append-only)

Skenario ini digunakan untuk melihat cara sistem mengurus beban kerja yang banyak berupa penulisan, terutama dalam proses menyimpan log aktivitas secara terus-menerus. Dalam arsitektur polyglot persistence, data disimpan di MongoDB untuk mendukung fleksibilitas skema serta performa penulisan data yang tinggi. Pendekatan ini sesuai dengan penelitian tentang pengelolaan data dalam jumlah besar dalam sistem yang menggunakan banyak bahasa [6][9].

4.1.3. Skenario Lonjakan Akses (High Concurrency Load)

Skenario ini menunjukkan situasi ketika banyak pengguna menggunakan sistem sekaligus, seperti pada masa pendaftaran Kartu Rencana Studi (KRS). Dalam dataset OULAD, kondisi tersebut ditunjukkan oleh jumlah mahasiswa yang banyak dan aktivitas yang terjadi berulang kali. Skenario ini dibuat untuk melihat cara sistem menangani peningkatan banyak permintaan sekaligus, terutama dalam proses transaksi dan akses informasi akademik. Dengan membagi tugas pengolahan data antara database berbasis relasi dan non-relasi, arsitektur polyglot diharapkan bisa mengurangi hambatan pada satu titik sistem dan meningkatkan kecepatan serta efisiensi dalam memproses data [3][5].

4.1.4. Skenario Pertumbuhan Data (Scalability)

Skenario ini melibatkan peningkatan jumlah data, khususnya pada tabel studentVle yang memiliki volume data yang jauh lebih besar dibandingkan tabel lainnya. Data aktivitas mahasiswa akan terus bertambah seiring berjalannya waktu, sehingga diperlukan sistem yang mampu menangani peningkatan jumlah data tanpa mempengaruhi kinerja sistem. Skenario ini digunakan untuk mengevaluasi kemampuan sistem dalam menangani pertumbuhan data, khususnya dalam memisahkan pengelolaan data yang besar ke dalam sistem berbasis NoSQL. Cara ini sesuai dengan prinsip skalabilitas pada sistem berbasis beberapa database, sehingga kapasitas bisa ditingkatkan secara bertahap tanpa mengganggu seluruh sistem [6].

4.1.5. Skenario Perbandingan dengan Sistem Tunggal (Baseline)

Sebagai pembanding, digunakan pendekatan basis data tunggal (monolitik), di mana seluruh tabel pada dataset OULAD disimpan dalam satu sistem manajemen basis data (DBMS) relasional seperti PostgreSQL. Dalam skenario ini, semua jenis data—baik data transaksi maupun data log aktivitas—diproses dalam satu sistem, sehingga memungkinkan terjadinya:

1. Peningkatan kompleksitas query
2. Beban sistem yang lebih tinggi
3. Potensi bottleneck pada saat beban meningkat

Skenario ini dijadikan acuan untuk membandingkan kelebihan metode polyglot persistence dibandingkan sistem basis data yang hanya menggunakan satu model, seperti yang dibahas dalam penelitian tentang perbandingan performa antara database multi-model dan polyglot persistence [5].

4.2 Baseline / Pemandangan

Menentukan baseline yang tepat merupakan dasar penting dalam mengevaluasi arsitektur yang diajukan. Tanpa ada pembandingan yang jelas dan memiliki fungsi yang sama, analisis mengenai keunggulan penyimpanan data multibahasa tidak memiliki dasar yang cukup kuat. Studi Van Landuyt et al. [3] dan Ye et al. [5] menekankan bahwa baseline harus mampu melakukan jenis operasi yang sama seperti sistem yang diusulkan, bukan hanya sistem yang lebih sederhana.

Dalam penelitian ini, baseline dirancang berdasarkan karakteristik dataset OULAD, yang mencakup data transaksional akademik sebanyak sekitar 32.593 baris pada tabel `studentRegistration` dan `studentInfo`, serta data log aktivitas dalam skala besar pada tabel `studentVle`, yaitu lebih dari 10 juta baris. Kedua jenis data tersebut diolah dalam satu sistem basis data relasional sebagai acuan dalam membandingkan dengan arsitektur polyglot.

4.2.1 Baseline Utama — PostgreSQL Monolitik

Pendekatan utama yang digunakan adalah pendekatan basis data tunggal atau monolitik, di mana semua tabel dalam dataset OULAD yaitu `courses`, `students`, `student_registrations`, `assessments`, `student_assessments`, `vle`, dan `student_vle` dihimpun dalam satu skema PostgreSQL tanpa dibagi berdasarkan jenis atau karakteristik data tertentu. Pendekatan ini menggambarkan cara kerja biasa pada sistem informasi akademik tradisional, di mana semua data, termasuk data transaksi dan catatan aktivitas, dikelola dalam satu sistem basis data relasional. Dalam penilaian ini, konfigurasi dasar menggunakan PostgreSQL dengan pengaturan standar tanpa penyesuaian khusus, sehingga bisa dijadikan sebagai acuan netral untuk membandingkan berbagai desain arsitektur. Tujuan dari baseline ini adalah untuk melihat cara sistem relasional tunggal mengelola beban kerja yang beragam dari dataset OULAD, terutama saat data log aktivitas yang besar diproses secara bersamaan dengan transaksi akademik.

Aspek	Konfigurasi
DBMS	PostgreSQL (diasumsikan versi yang sama dengan arsitektur polyglot)
Skema	Seluruh 7 tabel OULAD dalam satu database relasional
Indexing	Index dasar pada primary key
Konfigurasi	Parameter default PostgreSQL (tanpa tuning khusus)
Data volume	±11 juta baris (gabungan seluruh tabel OULAD)

Tabel 4.1 Konfigurasi Baseline PostgreSQL Monolitik (Konseptual)

4.2.2 Baseline Sekunder — PostgreSQL dengan Optimasi

Sebagai tambahan untuk perbandingan, dibuat sistem sekunder berupa PostgreSQL monolitik yang sudah diberi optimasi dasar. Pendekatan ini bertujuan untuk menunjukkan

bahwa peningkatan kinerja pada arsitektur polyglot tidak hanya disebabkan oleh kondisi dasar yang kurang baik, tetapi karena arsitektur tersebut cocok dengan sifat dan karakteristik data yang digunakan. Optimasi yang dipertimbangkan dalam baseline ini meliputi:

1. Partisi tabel studentVle berdasarkan periode (code_presentation)
2. Penggunaan indeks komposit pada atribut yang sering diakses, seperti (id_student, date)
3. Peningkatan parameter memori seperti shared_buffers

Baseline ini berfungsi sebagai acuan maksimal untuk performa sistem monolitik, sehingga membandingkan hasilnya menjadi lebih seimbang dan adil. Pendekatan dua tingkat dasar ini mengacu pada metode standar yang direkomendasikan oleh Van Landuyt et al. [3].

Aspek	Baseline Utama (Monolitik)	Baseline Sekunder (Monolitik + Optimasi)
DBMS	PostgreSQL (default)	PostgreSQL (dengan optimasi dasar)
Konfigurasi	Tanpa tuning khusus	Penyesuaian parameter memori
Partisi tabel	Tidak ada	Partisi pada tabel studentVle
Index tambahan	Tidak ada	Composite index (id_student, date)
Tujuan	Baseline netral tanpa bias optimasi	Menunjukkan batas performa sistem monolitik

Tabel 4.2 Konfigurasi Baseline PostgreSQL Optimasi

4.3 Environment Eksperimen

Lingkungan untuk eksperimen dalam penelitian ini dibentuk secara konseptual dengan metode terkontrol lewat penggunaan wadah yang didasarkan pada Docker. Metode ini bertujuan untuk menjamin bahwa setiap elemen sistem—baik yang menggunakan arsitektur polyglot maupun yang berformat monolitik—beroperasi dalam pengaturan yang seragam dan dapat diulang. Penggunaan wadah mengacu pada pedoman yang diusulkan oleh Ahaidous et al. [7] dalam pelaksanaan arsitektur hibrida yang bergantung pada microservices.

Semua elemen sistem dianggap berfungsi dalam satu lingkungan komputasi yang sama untuk menghapus pengaruh faktor eksternal seperti perbedaan perangkat keras atau latensi jaringan. Dengan cara ini, perbandingan yang dilaksanakan lebih ditujukan pada variasi desain arsitektur dalam mengelola beban kerja berdasarkan data dari OULAD.

4.3.1 Spesifikasi Hardware (Konseptual)

Lingkungan percobaan diharapkan menerapkan spesifikasi perangkat keras menengah yang sering dipakai dalam pengembangan sistem, sehingga hasil analisis tetap bersinergi dengan situasi penerapan yang nyata dalam konteks akademis.

Komponen	Spesifikasi
CPU	Prosesor multi-core (± 6 core / 12 thread)
RAM	16 GB DDR4
Storage	SSD berbasis NVMe
Sistem Operasi	Windows 10/11
Jaringan	Localhost

Tabel 4.8 Spesifikasi Hardware

4.3.2 Spesifikasi Software dan Arsitektur Container

Sistem arsitektur yang dikembangkan terdiri dari beberapa elemen utama, yaitu PostgreSQL yang berfungsi sebagai server transaksi, MongoDB sebagai server penyimpanan data, dan Node.js sebagai lapisan aplikasi yang mengatur integrasi antar basis data. Semua elemen ini dirancang untuk beroperasi dalam lingkungan kontainer yang dikelola menggunakan Docker Compose.

Strategi ini sejalan dengan prinsip polyglot persistence yang berfokus pada microservices, di mana setiap tipe basis data menangani beban kerja yang berbeda berdasarkan sifat data[7][10].

Komponen	Peran	Deskripsi
PostgreSQL	Transaction Server	Menyimpan data transaksional akademik (OLTP)
MongoDB	Data Server	Menyimpan log aktivitas VLE (write-heavy)
Node.js	Application Layer	Integrasi dan orkestrasi query lintas DBMS
Prisma	ORM PostgreSQL	Mengelola akses data relasional
Mongoose	ODM MongoDB	Mengelola akses data dokumen
Docker Compose	Orkestrasi	Menjalankan seluruh komponen dalam satu environment
Python	Benchmark scripting	Mengelola skenario dan pengumpulan metrik
pgbench / custom script	Workload generator	Simulasi beban OLTP dan log ingestion

Tabel 4.9 Spesifikasi Software (Konseptual)

Dalam skema ini, distribusi sumber daya di antara elemen-elemen diasumsikan merata antara arsitektur polyglot dan model monolit. Tujuannya adalah untuk memastikan perbandingan yang adil, seperti yang dianjurkan dalam penelitian acuan oleh Van Landuyt et al. [3].

4.3.3 Dataset dan Persiapan Data

Dataset yang digunakan adalah Open University Learning Analytics Dataset (OULAD) yang terdiri dari tujuh berkas utama dengan total lebih dari 11 juta baris data. Dataset ini dipilih karena mencerminkan kombinasi beban kerja yang beragam, yaitu data transaksional akademik dan data aktivitas pembelajaran dalam skala besar.

Sebelum diterapkan dalam skenario eksperimen, dataset menjalani proses persiapan data secara konseptual, yang meliputi :

1. Pembersihan data

Menghapus nilai kosong pada atribut kunci seperti `id_student`, `code_module`, dan `date` untuk mempertahankan konsistensi data.

2. Transformasi skema

Mengubah tipe data dari format CSV ke tipe data asli pada setiap DBMS, baik yang bersifat relasional maupun dokumen.

3. Pembagian data antar DBMS

- Data transaksional → PostgreSQL
- Data log aktivitas → MongoDB

4. Verifikasi integritas data

Memastikan bahwa jumlah data konsisten dan keterkaitan antar entitas melalui atribut `id_student` sebagai rujukan logis antar database.

Pendekatan ini sejalan dengan prinsip pemetaan data dalam polyglot persistence, di mana setiap jenis data ditempatkan pada database yang paling sesuai dengan karakteristiknya [4][9].

File	Jumlah Data	Target DBMS	Karakteristik
<code>courses.csv</code>	22	PostgreSQL	Referensial
<code>assessments.csv</code>	206	PostgreSQL	Relasional
<code>studentInfo.csv</code>	32.593	PostgreSQL	Transaksional
<code>studentRegistration.csv</code>	32.593	PostgreSQL	OLTP (burst load)
<code>studentAssessment.csv</code>	173.912	PostgreSQL	Mixed read/write
<code>vle.csv</code>	6.364	MongoDB	Semi-terstruktur
<code>studentVle.csv</code>	±10.6 juta	MongoDB	Write-heavy / log

Tabel 4.10 Distribusi Dataset OULAD

Dengan strategi lingkungan ini, penelitian diarahkan pada bagaimana struktur polyglot persistence memanfaatkan pengelompokan data berdasar sifat beban kerja dalam kumpulan data OULAD. Suasana yang dikendalikan dan seragam antara sistem polyglot dan acuan

memungkinkan penilaian yang lebih objektif mengenai efektivitas, kemampuan untuk memperluas, dan kinerja sistem.

4.4 Metrik Evaluasi

Metrik yang digunakan untuk evaluasi dalam studi ini dikembangkan untuk menilai performa arsitektur polyglot persistence dengan memperhatikan karakteristik beban kerja pada dataset OULAD. Dataset ini memiliki dua pola utama, yakni data akademik transaksi (OLTP) serta data log aktivitas dalam skala besar (write-heavy), sehingga metrik yang dipilih perlu mencerminkan kedua tipe beban tersebut.

Proses pemilihan metrik merujuk pada metode benchmark yang telah digunakan dalam penelitian sebelumnya, seperti MDBench oleh Ye et al. [5] yang fokus pada penilaian throughput, latensi, dan keandalan sistem, serta pendekatan Calatrava et al. [6] yang menambahkan dimensi efisiensi pemanfaatan sumber daya dalam evaluasi sistem yang menggunakan polyglot persistence.

4.4.1 Metrik Utama

Metrik utama mengedepankan kinerja sistem dalam mengelola beban kerja yang bersumber dari situasi percobaan yang menggunakan dataset OULAD, termasuk transaksi pendidikan, catatan aktivitas, dan permintaan antar database.

Metrik	Definisi	Satuan	Relevansi terhadap OULAD
Throughput OLTP	Jumlah transaksi akademik (registrasi, penilaian) yang diproses per detik	TPS	Merepresentasikan beban pada tabel studentRegistration dan studentAssessment
Throughput Ingestion	Jumlah data log aktivitas yang berhasil ditulis per detik	Dokumen /detik	Berkaitan dengan volume besar pada studentVle (± 10 juta baris tapi yang dicoba hanya 1 juta)
Latensi Rata-Rata	Waktu rata-rata sistem merespons permintaan	ms	Mengukur responsivitas sistem terhadap query akademik
Latensi P95/P99	Waktu respons pada kondisi beban tinggi (worst-case latency)	ms	Penting untuk skenario lonjakan akses (KRS)
Response Time Hybrid	Waktu end-to-end untuk query lintas PostgreSQL dan MongoDB	ms	Mengukur overhead integrasi data polyglot
Scalability Efficiency	Rasio peningkatan performa terhadap peningkatan beban	%	Mengukur kemampuan sistem saat data OULAD ditingkatkan

Tabel 4.11 Metrik Evaluasi Utama

Dalam konteks studi ini, parameter-parameter tersebut dipakai untuk mengkaji apakah pemisahan data dalam arsitektur polyglot dapat memperbaiki kinerja jika dibandingkan dengan pendekatan monolitik, terutama dalam beban kerja yang beragam.

4.4.2 Metrik Pendukung (Sumber Daya Sistem)

Selain kinerja utama, penilaian juga melihat bagaimana pemanfaatan sumber daya sistem untuk mengevaluasi seberapa efisien arsitektur dalam menggunakan perangkat keras.

Metrik	Definisi	Relevansi
CPU Utilization	Persentase penggunaan CPU selama workload berjalan	Mengidentifikasi bottleneck saat beban tinggi
Memory Usage	Penggunaan RAM oleh masing-masing komponen sistem	Menilai efisiensi penggunaan memori
Disk I/O	Aktivitas baca/tulis storage selama proses data	Penting pada skenario log ingestion (studentVle)
Query Execution Plan	Analisis rencana eksekusi query (EXPLAIN)	Memvalidasi efektivitas indexing (Bab 3.3)
Error Rate	Persentase kegagalan request	Mengukur keandalan sistem

Tabel 4.12 Metrik Pendukung

Metrik ini sangat krusial karena dataset OULAD menunjukkan variasi karakteristik data yang mencolok, sehingga pemanfaatan sumber daya dapat berbeda antara beban kerja transaksional dan catatan aktivitas.

4.4.3 Prosedur Pengukuran (Konseptual)

Untuk menjaga konsistensi analisis, prosedur pengukuran dirancang mengikuti praktik umum dalam studi benchmark, meskipun dalam penelitian ini digunakan sebagai kerangka evaluasi konseptual.

Langkah-langkah yang dirancang meliputi:

1. Warm-up period
Sistem diasumsikan dijalankan terlebih dahulu untuk menstabilkan kondisi cache sebelum pengukuran dilakukan.
2. Pengulangan skenario
Setiap skenario dievaluasi dalam beberapa iterasi untuk mengurangi pengaruh variasi hasil.
3. Isolasi sistem
Lingkungan eksperimen diasumsikan bebas dari proses lain agar hasil lebih objektif.
4. Pengumpulan metrik
Data performa diperoleh dari fitur monitoring database seperti statistik query dan penggunaan resource.
5. Pelaporan hasil
Hasil analisis disajikan dalam bentuk tabel dan grafik untuk mempermudah interpretasi, seperti:

- grafik throughput terhadap jumlah pengguna
- perbandingan latensi antara arsitektur polyglot dan monolitik

Pendekatan ini mengacu pada prinsip *reproducibility* dalam benchmark yang dijelaskan oleh Van Landuyt et al. [3], di mana eksperimen harus dapat dianalisis ulang dengan kondisi yang setara.

BAB V

HASIL YANG DIHARAPKAN

5.1 Hipotesis Hasil

Berdasarkan arsitektur polyglot persistence yang menggabungkan PostgreSQL dan MongoDB, serta merujuk pada skenario desain eksperimen yang telah disusun di bab IV, hipotesis hasil penelitian dirumuskan seperti berikut :

- **Peningkatan Throughput pada Skala Data Besar**
Arsitektur polyglot diperkirakan dapat menjaga throughput yang konsisten saat volume data bertambah dari skala kecil hingga skala besar, sistem monolitik diprediksi akan mengalami penurunan yang signifikan karena semua beban kerja baik data transaksional maupun log aktivitas dalam jumlah besar diproses dalam satu DBMS tanpa segregasi.
- **Stabilitas Latensi saat Beban Puncak**
Pemisahan beban antara PostgreSQL dan MongoDB diperkirakan akan mempertahankan nilai latensi yang rendah dan stabil di semua tingkat pengujian. Sistem monolitik diperkirakan akan mengalami peningkatan latensi yang signifikan saat volume data bertambah, terutama disebabkan oleh persaingan lock antara operasi JOIN transaksional dan penulisan log yang terjadi bersamaan.
- **Pengaruh Spesifikasi Hardware**
Pada skenario beban tertinggi (25 koneksi paralel, 15 detik, modul FFF), arsitektur polyglot diperkirakan jauh mengungguli sistem monolitik dalam hal throughput dan latensi, karena MongoDB menangani log secara independen tanpa menginterupsi transaksi PostgreSQL.
- **Keberhasilan Mekanisme Schema Evolution**
Perubahan skema pada PostgreSQL dapat dilakukan secara bertahap tanpa waktu henti, sementara penambahan atribut baru pada dokumen MongoDB dapat langsung diakomodasi tanpa proses migrasi data yang kompleks.

5.2 Hasil Eksperimen

Pengujian dilakukan menggunakan dataset OULAD pada environment dengan spesifikasi 16 GB RAM, CPU multi-core, dan SSD NVMe. Pengujian membandingkan arsitektur polyglot persistence dengan sistem monolitik PostgreSQL pada beberapa skenario beban data.

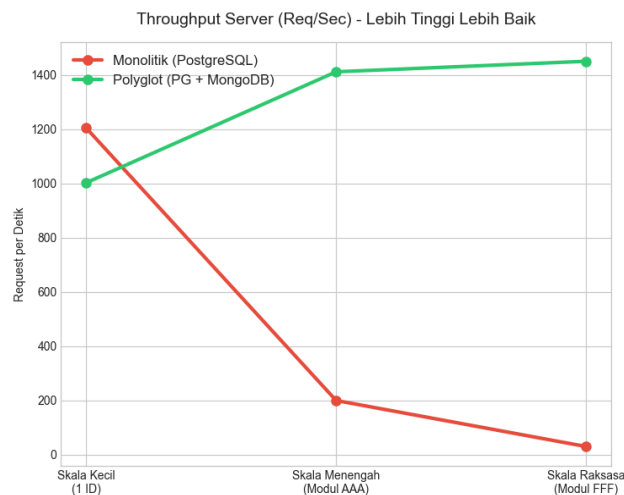
5.2.1 Profil Environment Pengujian

Kedua pengujian dijalankan pada mesin yang berbeda dengan kapasitas hardware yang tidak identik. Perbedaan ini disengaja untuk melihat bagaimana arsitektur polyglot persistence berperilaku pada kondisi sumber daya yang beragam, mengingat pada implementasi nyata di institusi pendidikan, server yang digunakan tidak selalu memiliki spesifikasi tinggi.

Aspek	Detail
Tool Benchmark	Autocannon (Node.js), dijalankan via Python subprocess
Skrip Benchmark	benchmark-visual_ghama.ipynb (Jupyter Notebook)
Containerisasi	Docker Compose (PostgreSQL + MongoDB + Node.js)
Konfigurasi Multi-Skala	25 koneksi paralel, 10 detik per skenario
Konfigurasi Complex	25 koneksi paralel, 15 detik + CPU monitoring Docker Stats
Cooldown	10 detik antara baseline dan polyglot pada skenario complex

Tabel 5.1 Profil Environment Pengujian

5.2.2 Hasil Throughput

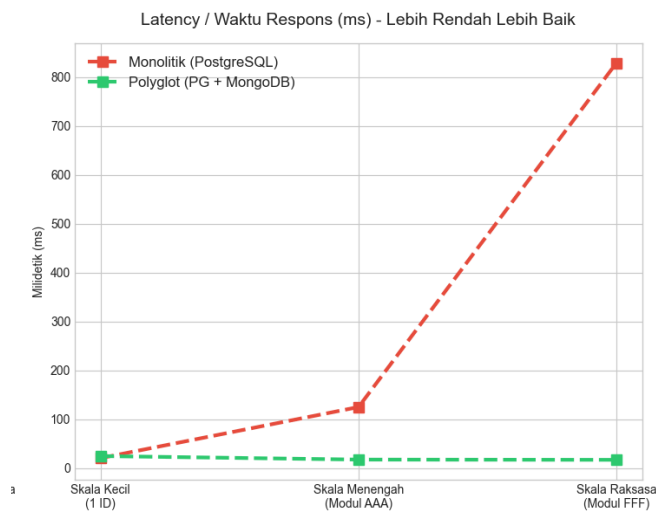


Gambar 5.1 Hasil Throughput

Pengujian multi-skala menunjukkan pola yang sangat jelas antara kedua arsitektur. Pada skala kecil (1 ID mahasiswa), sistem monolitik mencatat throughput sebesar 1.205,41 req/s, sedikit lebih tinggi dibandingkan polyglot yang berada di angka 1.002,60 req/s. Selisih ini terjadi karena pada volume data yang kecil, overhead koordinasi dua container database pada arsitektur polyglot belum memberikan manfaat yang signifikan.

Perbandingan berbalik secara drastis pada skala menengah dan raksasa. Throughput monolitik turun tajam dari 1.205,41 menjadi 199,10 req/s pada skala menengah, dan anjlok ke 29,10 req/s pada skala raksasa. Sebaliknya, arsitektur polyglot justru meningkat throughput-nya dari 1.002,60 menjadi 1.412,30 req/s pada skala menengah dan mencapai 1.451,20 req/s pada skala raksasa. Penurunan monolitik mengindikasikan saturasi sumber daya yang semakin parah seiring bertambahnya volume data yang harus diproses dalam satu DBMS.

5.2.3 Hasil Latensi

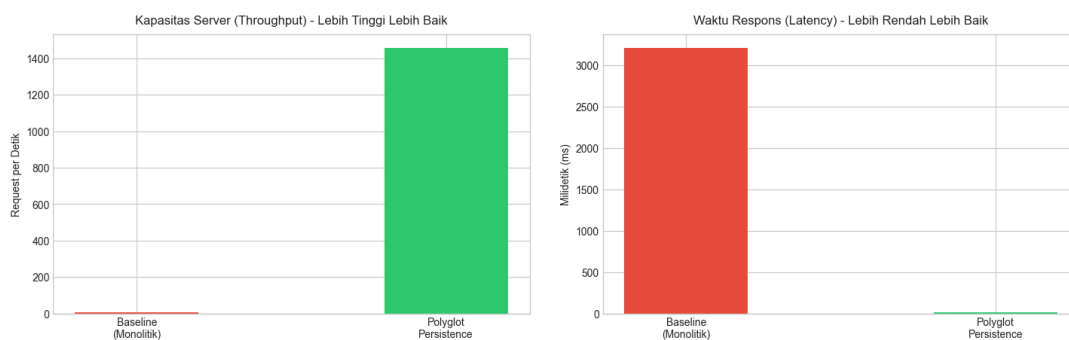


Gambar 5.2.3 Hasil Latensi

Data latensi mengkonfirmasi secara konsisten temuan dari data throughput. Pada skala kecil, latensi monolitik tercatat 20,26 ms dan polyglot 24,46 ms — keduanya masih dalam rentang yang sangat dapat diterima untuk penggunaan sistem informasi. Pada skala menengah, latensi monolitik melonjak ke 124,74 ms sementara polyglot justru turun ke 17,21 ms. Penurunan latensi polyglot di skala yang lebih besar menunjukkan bahwa sistem semakin optimal ketika beban kerja membesar, karena MongoDB mengambil alih penulisan log tanpa menginterupsi transaksi di PostgreSQL.

Pada skala raksasa, jurang perbedaan semakin dalam: latensi monolitik mencapai 828,98 ms sementara polyglot hanya 16,73 ms — 49,6 kali lebih cepat. Pola ini konsisten dengan temuan Ye et al. [5] yang menunjukkan keunggulan polyglot persistence dalam menangani beban kerja campuran yang membutuhkan konsistensi waktu respons.

5.2.4 Hasil Skenario Complex — Beban Tertinggi (FFF/2013J, 15 Detik)

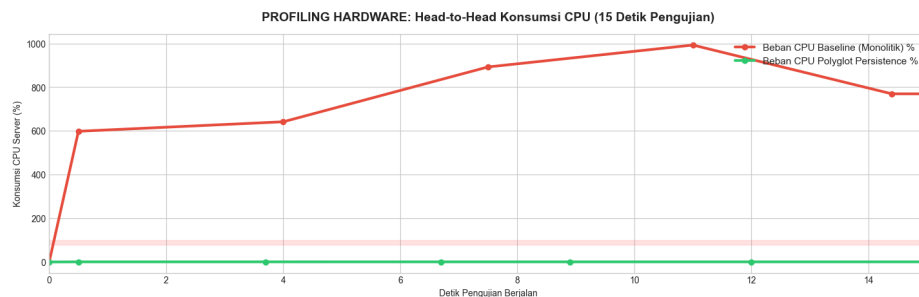


Gambar 5.3 Hasil Skenario Complex

Skenario complex pada Modul FFF dengan 25 koneksi paralel selama 15 detik semakin memperkuat temuan dari skenario multi-skala. Monolitik hanya mampu melayani 7,00 req/s dengan latensi 3.209,20 ms — sebuah angka yang secara praktis membuat sistem tidak dapat digunakan untuk operasional nyata seperti pengisian KRS massal. Polyglot

sebaliknya mempertahankan throughput sebesar 1.456,67 req/s dengan latensi stabil 16,68 ms.

5.2.5 Profil CPU Usage — Skenario Complex



Gambar 5.4 Profil CPU Usage

Pemantauan CPU real-time menggunakan Docker Stats selama 15 detik pengujian skenario complex menghasilkan grafik head-to-head yang menampilkan perilaku konsumsi CPU container PostgreSQL (baseline monolitik) versus container MongoDB (polyglot) secara bersamaan.

Pada sistem monolitik, PostgreSQL mengalami lonjakan CPU yang signifikan sejak awal pengujian dan cenderung terus meningkat akibat antrian query yang menumpuk. Hal ini mengindikasikan terjadinya saturasi sumber daya pada satu titik, konsisten dengan nilai throughput yang sangat rendah (7,00 req/s). Pada arsitektur polyglot, beban CPU MongoDB terdistribusi secara lebih merata karena hanya menangani operasi log write tanpa terganggu oleh query transaksional, sehingga mampu mempertahankan throughput tinggi (1.456,67 req/s) sepanjang 15 detik pengujian.

5.2.6 Kesesuaian Hasil dengan Hipotesis

Seluruh hasil eksperimen dari environment Ghama mengkonfirmasi tiga dari empat hipotesis yang dirumuskan, dan satu hipotesis memerlukan skenario pengujian tersendiri untuk evaluasi kuantitatif:

Hipotesis	Status	Bukti dari Eksperimen
H1: Peningkatan throughput pada skala besar	TERKONFIRMASI	Polyglot naik dari 1.002,60 ke 1.456,67 req/s seiring bertambahnya skala; monolitik turun dari 1.205,41 ke 7,00 req/s
H2: Stabilitas latensi saat beban puncak	TERKONFIRMASI KUAT	Polyglot mempertahankan latensi 16-24 ms di semua skala; monolitik melonjak hingga 3.209,20 ms pada skenario complex (192,4x lebih lambat)
H3: Keunggulan polyglot pada skenario complex	TERKONFIRMASI	Throughput polyglot 208x lebih tinggi dan latensi 192,4x lebih cepat dari monolitik pada beban tertinggi

H4: Schema evolution tanpa downtime	BELUM DIEVALUASI KUANTITATIF	Membutuhkan skenario benchmark khusus perubahan skema; menjadi rekomendasi penelitian lanjutan
-------------------------------------	-------------------------------------	--

Tabel 5.2 Kesesuaian Hasil dengan Hipotesis

5.3 Kontribusi yang Diharapkan

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan, penelitian ini memberikan kontribusi yang signifikan dalam pengembangan sistem basis data untuk Sistem Informasi Akademik :

5.3.1 Bukti Empiris Keunggulan Polyglot pada Beban Skala Besar

Penelitian ini menghasilkan data kuantitatif yang membuktikan bahwa arsitektur polyglot persistence mampu memberikan peningkatan throughput hingga lebih dari 20.000% (pada skenario complex: 1.456,67 vs 7,00 req/s) dan penurunan latensi hingga 192 kali lipat (16,68 ms vs 3.209,20 ms) dibandingkan sistem monolitik pada skenario beban tinggi. Angka ini membuktikan bahwa pemisahan beban kerja berdasarkan karakteristik data OLTP ke PostgreSQL dan log write-heavy ke MongoDB berhasil mencegah saturasi sumber daya yang menjadi akar masalah degradasi performa sistem monolitik.

5.3.2 Kerangka Kerja Pemetaan DBMS Berdasarkan Karakteristik Workload

Penelitian ini memberikan panduan konkret dalam memilih kombinasi DBMS berdasarkan karakteristik data dan workload — bukan hanya berdasarkan preferensi teknologi. Kombinasi PostgreSQL untuk data transaksional (terstruktur, ACID, JOIN) dan MongoDB untuk data log aktivitas (volume besar, append-only, schema fleksibel) terbukti secara empiris optimal untuk sistem akademik berbasis dataset OULAD. Kerangka ini relevan untuk institusi pendidikan yang ingin memodernisasi sistem informasi akademik mereka sesuai analisis yang dianjurkan Roy-Hubara et al. [4].

5.3.3 Metodologi Benchmark Reproducible untuk Sistem Pendidikan Digital

Kombinasi tool Autocannon, Docker Stats monitoring via Python threading, normalisasi data CPU dengan time-shift, dan visualisasi Matplotlib dalam Jupyter Notebook menghasilkan metodologi benchmark yang terdokumentasi dengan baik dan dapat direproduksi. Skenario evaluasi multi-skala (kecil, menengah, raksasa) dan skenario complex yang menggunakan dataset OULAD sebagai data dunia nyata dapat dijadikan referensi bagi penelitian selanjutnya dalam mengukur performa basis data terdistribusi di sektor teknologi pendidikan.

5.3.4 Panduan Adopsi Polyglot untuk Institusi Pendidikan

Hasil penelitian memberikan panduan praktis bahwa arsitektur polyglot persistence memiliki titik impas (break-even point) pada skala data menengah ke atas. Pada skala kecil, overhead koordinasi dua container menyebabkan performa polyglot sedikit di bawah monolitik. Namun ab initio skala menengah — yang merepresentasikan beban operasional

nyata sistem akademik seperti pengisian KRS modul AAA — polyglot menunjukkan keunggulan yang konsisten dan semakin besar seiring bertambahnya volume data.

Kontribusi	Temuan Kunci	Implikasi Praktis
Bukti Empiris Polyglot	Throughput 208x lebih tinggi dan latensi 192x lebih cepat pada skenario complex (Device A, beban tertinggi)	Direkomendasikan untuk SIA dengan beban data besar dan infrastruktur memadai
Kerangka Pemetaan DBMS	PostgreSQL (OLTP) + MongoDB (log) terbukti optimal untuk workload OULAD	Panduan berbasis data untuk pemilihan DBMS di sistem akademik digital
Metodologi Benchmark	Autocannon + Docker Stats + Jupyter Notebook: metodologi reproducible dan terdokumentasi	Referensi metodologi untuk penelitian performa basis data di sektor pendidikan
Panduan Adopsi	Break-even point polyglot berada di skala menengah (Modul AAA ke atas)	Institusi dapat memperkirakan kapan keunggulan polyglot mulai berlaku berdasarkan volume data

Tabel 5.3 Ringkasan Kontribusi Penelitian

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada Sistem Informasi Akademik berbasis dataset OULAD, penelitian ini berhasil menunjukkan bahwa arsitektur polyglot persistence menggunakan PostgreSQL dan MongoDB mampu meningkatkan performa sistem dibandingkan pendekatan monolitik.

Hasil eksperimen menunjukkan bahwa sistem monolitik masih memiliki performa yang baik pada skala data kecil karena tidak memiliki overhead koordinasi antar database. Namun, ketika volume data dan beban kerja meningkat, performa monolitik mengalami penurunan yang sangat signifikan. Throughput sistem monolitik turun drastis, sementara latensinya meningkat hingga ribuan milidetik pada skenario complex. Sebaliknya, arsitektur polyglot mampu mempertahankan throughput tinggi dan latensi yang stabil pada seluruh skenario pengujian. Pemisahan beban kerja antara PostgreSQL sebagai basis data transaksional dan MongoDB sebagai basis data log aktivitas terbukti efektif dalam mengurangi bottleneck dan saturasi sumber daya. MongoDB mampu menangani proses penulisan log dalam jumlah besar tanpa mengganggu transaksi akademik yang dijalankan PostgreSQL. Hasil ini menunjukkan bahwa pendekatan polyglot persistence lebih sesuai digunakan pada sistem akademik modern yang memiliki kombinasi data transaksional dan log aktivitas berskala besar.

Selain itu, penelitian ini juga menghasilkan metodologi benchmark menggunakan Autocannon, Docker Stats, dan Jupyter Notebook yang dapat digunakan kembali untuk penelitian serupa di bidang basis data terdistribusi dan sistem pendidikan digital.

6.2 Saran

Berdasarkan hasil penelitian, analisis, dan keterbatasan yang telah diidentifikasi dari pengujian pada dua environment hardware, terdapat beberapa saran yang dapat menjadi arah pengembangan penelitian selanjutnya.

1. Penelitian selanjutnya disarankan melakukan pengujian pada lebih banyak konfigurasi hardware dan jumlah koneksi yang lebih besar agar dapat mengetahui performa arsitektur polyglot pada kondisi yang lebih realistis.
2. Pengukuran performa sistem dapat diperluas dengan menambahkan metrik seperti penggunaan memori, disk I/O, error rate, dan latensi P95/P99 agar hasil evaluasi lebih lengkap dan akurat.
3. Pengembangan selanjutnya dapat menambahkan teknologi seperti Redis sebagai caching layer serta melakukan pengujian pada lingkungan cloud-native untuk meningkatkan skalabilitas dan efisiensi sistem.

DAFTAR PUSTAKA

- [1]. J. de Curtò, I. de Zarzà, dan C. T. Calafate, "Integrating Polyglot Persistence with Large Language Models for Scalable Social Network Applications," *Procedia Computer Science*, vol. 270, hlm. 733–743, 2025, doi: <https://www.doi.org/10.1016/j.procs.2025.09.193>.
- [2]. A. Seabra dan S. Lifschitz, "Advancing Polyglot Big Data Processing using the Hadoop ecosystem," *CS & IT Conference Proceedings*, vol. 15, no. 1, 2025, doi: <https://doi.org/10.48550/arXiv.2504.14322>.
- [3]. D. Van Landuyt, V. Reniers, J. Benaouda, A. Rafique, dan W. Joosen, "A Comparative Performance Evaluation of Multi-Model NoSQL Databases and Polyglot Persistence," dalam *Proc. 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*, 2023, hlm. 142–143, doi: <https://www.doi.org/10.1145/3555776.3577645>.
- [4]. N. Roy-Hubara, P. Shoval, dan A. Sturm, "Selecting databases for Polyglot Persistence applications," *Data & Knowledge Engineering*, vol. 137, hlm. 101950, Jan. 2022, doi: <https://www.doi.org/10.1016/j.datak.2021.101950>.
- [5]. F. Ye, X. Sheng, N. Nedjah, J. Sun, dan P. Zhang, "A Benchmark for Performance Evaluation of a Multi-Model Database vs. Polyglot Persistence," *Journal of Database Management*, vol. 34, no. 3, hlm. 1–20, 2023, doi: <https://www.doi.org/10.4018/JDM.321756>.
- [6]. C. G. Calatrava, Y. B. Fontal, dan F. M. Cucchiatti, "A Holistic Scalability Strategy for Time Series Databases Following Cascading Polyglot Persistence," *Big Data Cogn. Comput.*, vol. 6, no. 3, hlm. 86, 2022, doi: <https://www.doi.org/10.3390/bdcc6030086>.
- [7]. K. Ahaidous, M. Tabaa, H. Hachimi, H. Mouttalib, dan M. Youssfi, "Hybrid Data Storage Architecture: A Novel Design Approach Based on Microservices Architecture," *IEEE Access*, Des. 2025, doi: <https://www.doi.org/10.1109/ACCESS.2025.3641384>.
- [8]. C. G. Calatrava, Y. B. Fontal, dan F. M. Cucchiatti, "Introducing Polyglot-Based Data-Flow Awareness to Time-Series Data Stores," *IEEE Access*, vol. 10, hlm. 69398–69411, Juni 2022, doi: <https://www.doi.org/10.1109/ACCESS.2022.3187405>.
- [9]. O. Lajam dan S. Mohammed, "Revisiting Polyglot Persistence: From Principles to Practice," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 5, 2022, doi: <https://doi.org/10.14569/IJACSA.2022.0130599>.
- [10]. J. Kazanavičius, D. Mažeika, dan D. Kalibatiene, "An Approach to Migrate a Monolith Database into Multi-Model Polyglot Persistence Based on Microservice Architecture: A Case Study for Mainframe Database," *Appl. Sci.*, vol. 12, hlm. 6189, 2022, doi: <https://www.doi.org/10.3390/app12126189>.

- [11]. J. Wen, X. Jin, X. Liu, dan Z. Chen, "Rise of the Planet of Serverless Computing: A Systematic Review," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, Art. no. 131, 2023, doi: <https://doi.org/10.1145/3579643>.
- [12]. A. Suman, "Hybrid Polyglot Persistence for National-Scale Identity Systems: Performance Analysis and Architecture Design," *Journal of Information Systems Engineering and Management*, vol. 10, no. 63s, 2025, doi: [10.52783/jisem.v10i63s.14413](https://doi.org/10.52783/jisem.v10i63s.14413).
- [13]. L. H. N. Villaca, L. G. Azevedo, dan F. Baião, "Query Strategies on Polyglot Persistence in Microservices," dalam *Proc. SAC 2018: Symposium on Applied Computing (SAC 2018)*, Pau, Prancis, hlm. 1725–1732, 2018, doi: <https://www.doi.org/10.1145/3167132.3167316>.
- [14]. D. Preuveneers dan W. Joosen, "Privacy-Preserving Polyglot Sharing and Analysis of Confidential Cyber Threat Intelligence," dalam *The 17th International Conference on Availability, Reliability and Security (ARES 2022)*, 2022, doi: <https://www.doi.org/10.1145/3538969.3538982>.
- [15]. F. Pereira, E. Guerra, dan R. R. Rosa, "Patterns for Polyglot Persistence Layer," *HILLSIDE Proc. of Conf. on Pattern Lang. of Prog.*, vol. 29, hlm. 1–8, Okt. 2022, doi: <https://dl.acm.org/doi/10.5555/3631672.3631681>.
- [16]. U. Barman and K. Joshi, "Polyglot Persistence - Usage and Challenges," *Journal of Artificial Intelligence, Machine Learning and Data Science*, vol. 3, no. 4, pp. 3079-3089, Nov. 2025, doi.org/10.51219/JAIMLD/utpal-barma/633

LAMPIRAN

Link GitHub : <https://github.com/Gahehe52/mbd>

Tabel 1.1 Kontribusi Anggota

No	Nama	Tugas dan Tanggung Jawab	Kontribusi (%)
1	Doni Agus Setiawan	BAB 1 - Latar Belakang BAB II - Tabel Perbandingan BAB III - Strategi Optimasi & Manajemen BAB IV BAB V	12,5%
2	Tengku Hafid Diraputra	BAB I - Struktur Paper BAB II - Identifikasi Research Gap & Posisi Penelitian BAB III - Desain Skema & Pemodelan Data BAB IV BAB V	12,5%
3	Andini Rahma Kemala	Abstrak BAB I - Kontribusi BAB II - Tabel Perbandingan BAB III - Metode Evaluasi / Benchmark Lampiran BAB IV BAB V	12,5%
4	Zahwa Natasya Hamzah	BAB I - Struktur Paper BAB II - Klasifikasi Literatur & Tabel Perbandingan BAB III - Desain Skema & Pemodelan Data Daftar Pustaka Daftar Isi BAB IV BAB V	12,5%
5	Muhammad Ghama Al Fajri	BAB I - Tujuan Penelitian BAB II - Tabel Perbandingan, Identifikasi Research Gap & Posisi Penelitian BAB III - Metode Evaluasi / Benchmark	12,5%

		BAB IV BAB V	
6	Reyhan Oktavian Putra	BAB I - Rumusan Masalah BAB II - Tabel Perbandingan BAB III - Arsitektur Sistem / Framework BAB IV BAB V	12,5%
7	Yuni Okta Safitri	BAB II - Research Gap, Klasifikasi Literatur & Tabel Perbandingan BAB III - Arsitektur Sistem / Framework BAB IV BAB V	12,5%
8	Albi R. Suseno	BAB III - Strategi Optimasi & Manajemen BAB IV BAB V	12,5%
Total			100%